

DBMS – REVIEW AND RATING TABLE MANAGEMENT SYSTEM

AJIO E-COMMERCE DATABASE MANAGEMENT SYSTEM

1. DATABASE DESIGN

1.1 REVIEW TABLE

<i>Attribute</i>	<i>Data Type</i>	<i>Description</i>
Review_ID	NUMBER(5)	Unique review ID
Customer_ID	NUMBER(5)	Customer ID
Product_ID	NUMBER(5)	Product ID
Rating	NUMBER(1)	Product rating from 1 to 5
Review_Text	VARCHAR2(500)	Customer review
Review_Date	DATE	Date of review

1.2 RATING TABLE

<i>Attribute</i>	<i>Data Type</i>	<i>Description</i>
Rating_ID	NUMBER(5)	Unique rating ID
Customer_ID	NUMBER(5)	Customer ID
Product_ID	NUMBER(5)	Product ID
Rating	NUMBER(1)	Product rating from 1 to 5
Rating_Date	DATE	Date of rating

1.3 Relation between RATING, REVIEW and CUSTOMER1

- **CUSTOMER1 → REVIEW:** One customer can give reviews.
- **CUSTOMER1 → RATING:** One customer can give ratings.
- **REVIEW → RATING :** Both are connected through CUSTOMER_ID and PRODUCT_ID.
- **CUSTOMER_ID** identifies which customer gave the review and rating.
- **PRODUCT_ID** identifies which product was reviewed and rated.

2. TABLE CREATION

2.1 CREATING REVIEW TABLE

```
SQL> CREATE TABLE REVIEW (  
2     REVIEW_ID NUMBER(5) PRIMARY KEY,  
3     CUSTOMER_ID NUMBER(5),  
4     PRODUCT_ID NUMBER(5),  
5     RATING NUMBER(1) CHECK (RATING BETWEEN 1 AND 5),  
6     REVIEW_TEXT VARCHAR2(500) NOT NULL,  
7     REVIEW_DATE DATE,  
8     CONSTRAINT FK_REVIEW_CUSTOMER  
9         FOREIGN KEY (CUSTOMER_ID)  
10        REFERENCES customer1(customer_id)  
11 );
```

Table created.

2.2 CREATING RATING TABLE

```
SQL> CREATE TABLE RATING (  
  2     RATING_ID NUMBER(5) PRIMARY KEY,  
  3     CUSTOMER_ID NUMBER(5),  
  4     PRODUCT_ID NUMBER(5),  
  5     RATING NUMBER(1) CHECK (RATING BETWEEN 1 AND 5),  
  6     RATING_DATE DATE,  
  7     CONSTRAINT FK_RATING_CUSTOMER  
  8         FOREIGN KEY (CUSTOMER_ID)  
  9         REFERENCES customer1(customer_id)  
 10 );
```

Table created.

3. VALUES INSERTION

3.2 INSERTING VALUES TO REVIEW TABLE

```
SQL> INSERT INTO REVIEW VALUES  
  2  (1,101,101,5,'Excellent product and very good quality.',DATE '2026-08-01');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW VALUES  
  2  (2,102,102,4,'Good product with nice quality.',DATE '2026-08-02');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW VALUES  
  2  (3,103,103,5,'Very comfortable and worth the price.',DATE '2026-08-03');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW VALUES  
  2  (4,104,104,3,'Product is good but delivery was late.',DATE '2026-08-04');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW VALUES  
  2  (5,105,105,4,'Good quality and attractive design.',DATE '2026-08-05');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW VALUES  
  2  (6,106,106,5,'Excellent product. I really liked it.',DATE '2026-08-06');  
  
1 row created.
```

3.2 INSERTING VALUES TO RATING TABLE

```
SQL> INSERT INTO RATING VALUES  
2 (1,101,101,5,DATE '2026-08-01');
```

1 row created.

```
SQL>  
SQL> INSERT INTO RATING VALUES  
2 (2,102,102,4,DATE '2026-08-02');
```

1 row created.

```
SQL>  
SQL> INSERT INTO RATING VALUES  
2 (3,103,103,5,DATE '2026-08-03');
```

1 row created.

```
SQL>  
SQL> INSERT INTO RATING VALUES  
2 (4,104,104,3,DATE '2026-08-04');
```

1 row created.

```
SQL>  
SQL> INSERT INTO RATING VALUES  
2 (5,105,105,4,DATE '2026-08-05');
```

1 row created.

```
SQL>  
SQL> INSERT INTO RATING VALUES  
2 (6,106,106,5,DATE '2026-08-06');
```

1 row created.

4. DISPLAY RATING TABLE

```
SQL> SELECT * FROM RATING;
```

RATING_ID	CUSTOMER_ID	PRODUCT_ID	RATING	RATING_DA
1	101	101	5	01-AUG-26
2	102	102	4	02-AUG-26
3	103	103	5	03-AUG-26
4	104	104	3	04-AUG-26
5	105	105	4	05-AUG-26
6	106	106	5	06-AUG-26
7	107	107	4	07-AUG-26
8	108	108	3	08-AUG-26
9	109	109	5	09-AUG-26
10	110	110	4	10-AUG-26

```
10 rows selected.
```

5. DISPLAYING REVIEW TABLE

```
SQL> SELECT * FROM REVIEW;
```

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING
REVIEW_TEXT			
REVIEW_DA			
1	101	101	5
Excellent product and very good quality.			
01-AUG-26			

2	102	102	4
Good product with nice quality.			
02-AUG-26			

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING
REVIEW_TEXT			
REVIEW_DA			
3	103	103	5
Very comfortable and worth the price.			
03-AUG-26			

4	104	104	3
Product is good but delivery was late.			

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING
REVIEW_TEXT			
REVIEW_DA			
04-AUG-26			
5	105	105	4
Good quality and attractive design.			
05-AUG-26			

6	106	106	5
REVIEW_ID CUSTOMER_ID PRODUCT_ID RATING			
REVIEW_TEXT			

6. PRODUCT WISE RATING ANALYSIS

```
SQL> SELECT product_id,  
2         AVG(rating) AS average_rating,  
3         COUNT(rating) AS total_ratings  
4 FROM RATING  
5 GROUP BY product_id;
```

PRODUCT_ID	AVERAGE_RATING	TOTAL_RATINGS
101	5	1
102	4	1
103	5	1
104	3	1
105	4	1
106	5	1
107	4	1
108	3	1
109	5	1
110	4	1

10 rows selected.

7. AVERAGE PRODUCT RATING

```
SQL> SELECT p.product_name,  
2         ROUND(AVG(r.rating), 2) AS average_rating  
3 FROM RATING r  
4 JOIN AJIOPProduct p  
5 ON r.product_id = p.product_id  
6 GROUP BY p.product_name  
7 HAVING AVG(r.rating) >= 4;
```

PRODUCT_NAME

AVERAGE_RATING

Men Casual Shirt
5

Women Kurti
4

Running Shoes
5

8. HIGHLY RATED PRODUCTS

```
SQL> SELECT p.product_name,  
2         ROUND(AVG(r.rating), 2) AS average_rating  
3 FROM RATING r  
4 JOIN AJIOPProduct p  
5 ON r.product_id = p.product_id  
6 GROUP BY p.product_name  
7 HAVING AVG(r.rating) >= 4;
```

PRODUCT_NAME

AVERAGE_RATING

Men Casual Shirt

5

Women Kurti

4

Running Shoes

5

9. RETRIVE PRODUCT REVIEW DETAILS

```
SQL> SELECT r.review_id,  
2         c.first_name,  
3         p.product_name,  
4         r.rating,  
5         r.review_text,  
6         r.review_date  
7 FROM REVIEW r  
8 JOIN customer1 c  
9 ON r.customer_id = c.customer_id  
10 JOIN AJIOPProduct p  
11 ON r.product_id = p.product_id;
```

REVIEW_ID FIRST_NAME

PRODUCT_NAME

RATING

REVIEW_TEXT

REVIEW_DA

2 Rahul

Women Kurti

4

Good product with nice quality.

02-AUG-26

3 Ananya

Running Shoes

5

Very comfortable and worth the price.

03-AUG-26

1 Pooja

Men Casual Shirt

5

Excellent product and very good quality.

01-AUG-26